

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего образования

«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики

Кафедра вычислительных технологий

Отчет

по лабораторной работе №3 по курсу

«МЕТОДЫ ПОИСКОВОЙ ОПТИМИЗАЦИИ»

Работу выполнили

Студенты группы ФИ31/2:

Костров А.А. и Долгов М.И.

Преподаватель:

Нигодин Е.А.

Краснодар 2026

Тема работы: «Генетический алгоритм»

Цель работы: изучить методы безусловной поисковой оптимизации глобальных экстремумов функций с помощью генетических алгоритмов

Основные понятия

Хромосома — вектор (или строка) из каких-либо чисел.

Индивидуум (генетический код, особь) — набор хромосом (вариант решения задачи).

Кроссинговер (кроссовер) — операция, при которой две хромосомы обмениваются своими частями.

Мутация — случайное изменение одной или нескольких позиций в хромосоме.

Популяция — совокупность индивидуумов.

Пригодность (приспособленность) — критерий или функция, экстремум которой следует найти.

Шаги алгоритма

Основные принципы работы ГА заключены в следующей схеме:

1. Генерируем начальную популяцию из n хромосом.
2. Вычисляем для каждой хромосомы ее пригодность.
3. Выбираем пару хромосом-родителей с помощью одного из способов отбора.
4. Проводим кроссинговер двух родителей с вероятностью p_c , производя двух потомков.

5. Проводим мутацию потомков с вероятностью p_m .
6. Повторяем шаги 3–5, пока не будет сгенерировано новое поколение популяции, содержащее n хромосом.
7. Повторяем шаги 2–6, пока не будет достигнут критерий окончания процесса (им может служить заданное количество поколений или такое состояние популяции, когда все строки популяции почти одинаковы и находятся в области некоего экстремума).

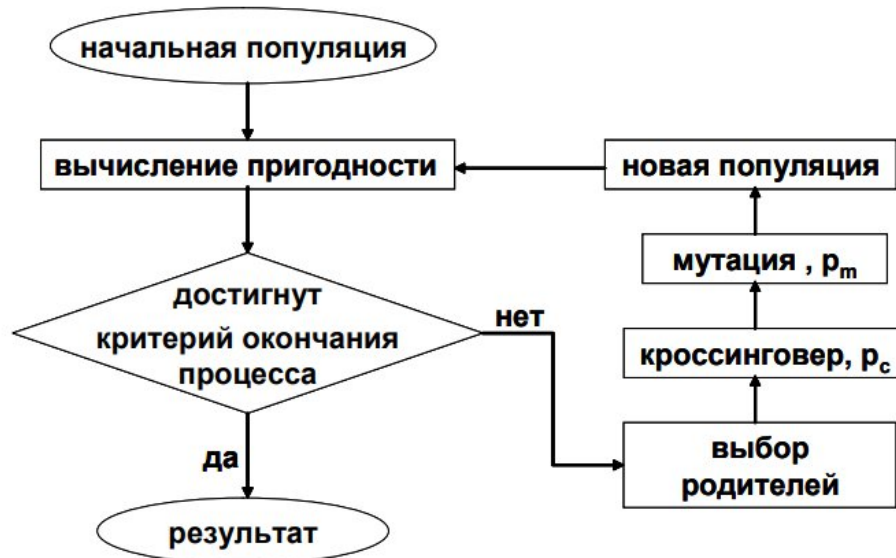


Рисунок 1 – Схема простого ГА

Особенности реализации генетического алгоритма

В качестве способа отбора был выбран метод руле-ки.

$$P(i) = \frac{f(i)}{\sum_{i=1}^N f(i)},$$

В рулетке участвуют все особи одновременно, выбираемые единожды случайно с вероятностью, пропорциональной их пригодности относительно всей популяции. Особи сравниваются по значению функции в точках, соответствующих хромосомам особи.

В качестве способа рекомбинации была выбрана линейная рекомбинация с параметрами $d = 0,25$ и $\alpha \in [-0,25; 1,25]$. Таким образом числовой интервал значений хромосом потомков, который должен содержать значения хромосом родителей: $[-d, 1 + d]$, то есть $[-0,25; 1, 25]$, а потомки создаются по следующему правилу:

$$\text{Потомок} = \text{Родитель 1} + \alpha \cdot (\text{Родитель 2} - \text{Родитель 1})$$

Для каждого создаваемого потомка выбирается отдельный множитель α — он применяется для вычисления всех хромосом нового потомка. Для следующего потомка, этот множитель будет отличаться.

В качестве механизма мутации был выбран вариант «Мутация для вещественных особей», где мутация происходит с вероятностью p_m и реализуется как: новая переменная = старая переменная $\pm \alpha \cdot \delta$, где $\pm$ выбираются с равной вероятностью, $\alpha = 0.5$ и $\delta = \sum_{i=n}^m a(i) * 2^{-i}$

На рисунке 2 представлен пример работы реализованного генетического алгоритма:

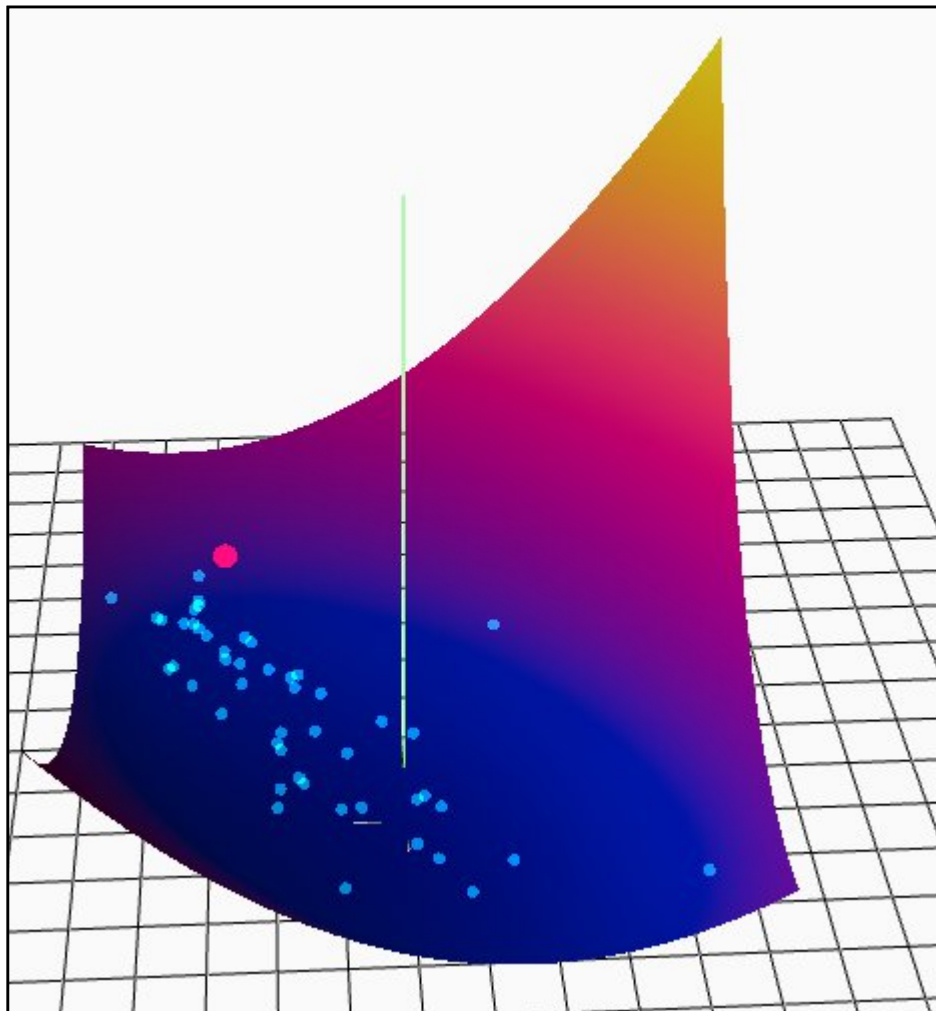


Рисунок 2 – Пример работы ГА на функции Матьяса

Вывод: в ходе работы были изучены различные методы безусловной поисковой оптимизации с использованием ГА, реализован ГА.

Листинг программы:

```
import numpy as np

class GeneticAlgorithm:
    """Класс с алгоритмом генетической оптимизации"""
    def __init__(self, func, x_range, y_range, population_size, survived_rate, mutation_param, generations, **func_params):
        self._func = func
        self._func_params = func_params
        self._life_space = (x_range, y_range)
        self._population_size = population_size
        self._survived_rate = survived_rate
        self._mutation_param = mutation_param
        self._generations = generations
        self._population = self._initialize_population()

    def get_population_size(self):
        """Возвращает размер популяции"""
        return self._population_size

    def get_population(self):
        """Возвращает текущую популяцию"""
        return self._population

    def set_population(self, population):
        """Устанавливает новую популяцию"""
        self._population = population

    def parent_selection(self):
        """Выбор родителей для скрещивания методом рулетки с функцией приспособленности по нормализованным значениям функции"""
```

```

        parent_values = [self._func(*parent, **self._func_params) for parent
in self._population]

        F_min = min(parent_values)

        F_max = max(parent_values)

        fitness_values = [self._fitness(F_i, F_min, F_max) for F_i in par-
ent_values]

        total_fitness = sum(fitness_values)

        if total_fitness == 0:

            probabilities = [1 / self._population_size] * self._popula-
tion_size

        else:

            probabilities = [f / total_fitness for f in fitness_values]

        indices = np.random.choice(

            len(self._population),

            size=self._survived_rate,

            p=probabilities,

            replace=False

        )

        survived_parents = [self._population[i] for i in indices]

        return survived_parents

def crossover(self, parent1, parent2) -> tuple:

    """Скрещивание двух родителей для получения потомка"""

    d = 0.25

    alpha = np.random.uniform(-d, 1+d)

    return (

        (

            parent1[0] + alpha * (parent2[0] - parent1[0]),

            parent1[1] + alpha * (parent2[1] - parent1[1])

        ),

    )

```

```

        (
            parent2[0] + alpha * (parent1[0] - parent2[0]),
            parent2[1] + alpha * (parent1[1] - parent2[1])
        )
    )

def mutation(self, individual, m = 20) -> tuple:
    """Мутация особи"""
    x, y = individual
    if np.random.rand() < self._mutation_param:
        alpha = 0.5
        x_ratio = (self._life_space[0][1] - self._life_space[0][0])
        y_ratio = (self._life_space[1][1] - self._life_space[1][0])
        mult = np.random.choice([-1, 1])
        delta = sum(
            (1 if np.random.rand() < 1 / m else 0) * 2**(-i)
            for i in range(1, m + 1)
        )
        x += mult * alpha * x_ratio * delta
        y += mult * alpha * y_ratio * delta
        x = max(self._life_space[0][0], min(x, self._life_space[0][1]))
        y = max(self._life_space[1][0], min(y, self._life_space[1][1]))
    return x, y

@staticmethod
def _fitness(F_i, F_min, F_max):
    """Нормализованная функция приспособленности для минимизации"""
    if F_max == F_min:
        return 1.0 # все особи одинаковы

```



```

        return (F_max - F_i) / (F_max - F_min)

    def _initialize_population(self):
        """Инициализирует начальную популяцию"""
        population = []
        for _ in range(self._population_size):
            x = np.random.uniform(self._life_space[0][0],
self._life_space[0][1])
            y = np.random.uniform(self._life_space[1][0],
self._life_space[1][1])
            population.append((x, y))
        return population

def genetic_algorithm(func, x_range: tuple, y_range: tuple, pop_size: int,
survived_size: int, p_mutation: float, generations: int = 100,
clear_view=None, **func_params):
    """
    Метод генетической оптимизации для квадратичного программирования
    Реализует алгоритм из лабораторной работы №3
    """
    alg_model = GeneticAlgorithm(func, x_range, y_range, pop_size, sur-
vived_size, p_mutation, generations, **func_params)
    population = alg_model.get_population()
    yield sorted(population, key=lambda x: alg_model._func(*x,
**alg_model._func_params))
    for _ in range(generations):
        survived = alg_model.parent_selection()
        while len(survived) < alg_model.get_population_size():
            idx1, idx2 = np.random.choice(len(survived), size=2, re-
place=False)
            parent1, parent2 = survived[idx1], survived[idx2]
            child1, child2 = alg_model.crossover(parent1, parent2)
            child1 = alg_model.mutation(child1)

```

```
        child2 = alg_model.mutation(child2)
        survived.append(child1)
        if len(survived) < alg_model.get_population_size():
            survived.append(child2)
    population = survived
    alg_model.set_population(population)
    if clear_view:
        clear_view()
    yield sorted(population, key=lambda x: alg_model._func(*x,
**alg_model._func_params))
```